

LangSmith介绍



官方文档: <https://docs.smith.langchain.com/>

一、没有 LangSmith 会面临哪些痛点？



在生产环境中部署 LLM 应用后，你是否曾遇到这些问题？

1. 调试困难，黑盒运行

- LLM 调用缺乏可观测性，Prompt 输入什么、模型输出了什么、消耗了多少 Token，开发者完全看不到
- 当 AI 应用输出错误结果时，难以定位是 Prompt 问题、模型问题还是业务逻辑问题
- 线上问题只能靠"猜"，排查效率极低

2. 缺乏系统化的评估手段

- 依赖人工 Review 来评估输出质量，成本高、效率低
- 没有量化指标，无法客观衡量应用在不同场景下的表现
- 版本迭代时没有回归测试，不知道改动是否引入了新问题

3. 性能与成本不可控

- 无法追踪 Token 消耗，月底收到账单时才发现超支
- 不知道哪些请求慢、为什么慢，缺乏性能瓶颈的可见性
- 无法区分不同用户、不同场景的调用量，优化无从下手

4. 知识碎片化，难以复用

- 每次出问题都要从头分析，团队经验无法沉淀
- 好的 Prompt 和 Bad Case 散落在各个文档或聊天记录里
- 新成员加入时没有可参考的历史调试记录

5. 多模型/多应用管理混乱

- 切换模型或引入新 Provider 时，没有统一的观测入口

- 同一个应用在多个渠道（网页、APP、API）表现不一致，难以排查

二、LangSmith 是什么？解决什么问题？

 LangSmith 是一个 AI 应用可观测性平台，帮助开发者更好地调试、评估和监控 LLM 应用。

LangSmith 由 **LangChain** 团队开发，于 2023 年正式发布，专为基于大语言模型的应用设计。

核心价值

- **可视化**：将 AI 应用的每一次调用（Prompt、Response、Token、Latency）完整记录并展示
- **可评估**：提供自动化评估流水线，支持定义测试数据集和量化指标
- **可监控**：实时追踪应用健康状况，性能和成本一目了然
- **可协作**：团队成员可以共享调用记录、标注数据、评估结果

三、LangSmith 核心功能详解

功能一：Trace（链路追踪）

 Trace 是 LangSmith 最核心的功能，相当于 AI 应用的"Replay 功能"。

每一次 LLM 调用都会被完整记录：

记录内容	说明
输入 Prompt	用户发送的原始 Prompt
输出 Response	模型的完整回复
Token 消耗	输入/输出 Token 数量及费用
响应延迟	从发请求到收到回复的耗时
调用链	如果是 Chain/Agent，显示完整的调用步骤
元数据	模型版本、温度、时间戳等

使用场景：线上出现 Bad Case 时，可以直接找到那条调用记录，重放整个过程，精确定位问题。

功能二：Evaluation（自动化评估）



LangSmith 支持定义评估标准和测试数据集，实现 AI 输出的自动化评分。

评估方式：

- **人工评估：**人工标注，定义"好/坏/中等"
- **AI 辅助评估：**用另一个 LLM 自动判断输出质量
- **基于规则的评估：**Regex、关键词匹配等硬规则
- **测试数据集 (Datasets)：**
 - 上传标准输入和期望输出
 - 每次代码变更后自动跑回归测试
 - 对比不同模型、不同 Prompt 的表现差异

典型应用：

在发布新版本 Prompt 之前，先用测试数据集跑一遍，自动化验证输出质量是否达标，避免"发布后才发现效果变差"的问题。

功能三：Monitoring（监控与告警）



实时监控 AI 应用的关键指标，设置告警规则。

核心指标：

- **调用量：**总请求数、按用户/应用/模型分组的请求量
- **Token 消耗：**日/月累计用量和费用
- **响应延迟：**P50、P95、P99 延迟分布
- **错误率：**模型拒绝、API 超时等异常比例
- **质量指标：**评估分数趋势（需配合 Evaluation）
- **告警功能：**
 - 错误率超过阈值 → 发送通知
 - Token 消耗逼近预算上限 → 告警提醒
 - 评估分数突然下降 → 触发回归告警

功能四：Datasets & Runs（数据集管理）



LangSmith 提供结构化的数据集管理能力，支持导入、标注和复用测试数据。

核心操作：

- **创建数据集**：手动上传或从生产环境的 Trace 中批量导入
- **添加标注**：对输入输出添加人工标签或期望输出
- **关联 Runs**：将数据集的每条数据关联到具体的调用记录
- **版本管理**：支持数据集版本，方便对比不同测试集下的表现

功能五：Prompt 管理与版本控制



LangSmith 内置 Prompt Registry，支持对 Prompt 模板进行版本化管理。

- **版本历史**：每次修改都有记录，可回滚到任意版本
- **A/B 测试**：同时部署多个版本的 Prompt，对比效果
- **变量绑定**：将 Prompt 模板与具体数据集/应用关联

功能六：团队协作



LangSmith 支持多成员协作，适合团队开发 AI 应用。

- **分享 Trace**：通过链接分享某次调用记录给队友
- **共享数据集**：团队成员共同维护测试数据集
- **权限控制**：不同的成员可以设置不同的访问权限
- **审计日志**：记录谁在什么时候做了什么操作

四、总结



LangSmith 补齐了 LLM 应用开发中最缺失的一环——可观测性。

痛点	LangSmith 解决方案

调试困难	Trace 重放每一次调用
缺乏评估	Evaluation + Datasets 自动化评分
性能成本不透明	Monitoring 实时追踪
经验难沉淀	数据集管理 + Prompt 版本控制
团队协作难	共享 Trace 和协作功能

对于正在构建或维护 AI 应用的团队，LangSmith 是提升研发效率和产品质量的利器。

五、补充说明：LangSmith 技术架构与集成方式



了解 LangSmith 的底层设计，有助于更好地集成和使用。

5.1 与 LangChain 的深度集成

LangSmith 与 LangChain 是天然搭档：

- LangChain Python / JS SDK 内置 LangSmith 追踪支持，只需一个环境变量即可开启
- 自动将 LangChain 的 Chain、Agent、Tool、Retriever 等组件的执行过程记录为嵌套的 Trace
- 支持 LangChain Expression Language (LCEL) 的每一步可视化

快速接入 (LangChain Python)：

代码块

```
1  # 设置环境变量
2  import os
3  os.environ["LANGCHAIN_TRACING_V2"] = "true"
4  os.environ["LANGCHAIN_API_KEY"] = "ls_..."
5
6  # 之后所有 LangChain 调用自动被追踪
7  from langchain_openai import ChatOpenAI
8  llm = ChatOpenAI()
9  # 后续的 chain 调用会自动记录到 LangSmith
10
```

5.2 支持非 LangChain 应用

LangSmith 并不强制依赖 LangChain，也可以接入任意 LLM 应用：

- **OpenAI SDK**：直接通过 LangSmith 的装饰器或回调函数接入

- **自定义 LLM 调用**：通过 LangSmith Python/JS SDK 的 `Client` 手动创建 Run
- **Anthropic、HuggingFace、Azure OpenAI 等**：均通过上述方式支持

5.3 Runs 的数据结构

LangSmith 将每一次调用抽象为 **Run**（也称 **Trace**），核心数据结构：

字段	说明
inputs	用户输入（如 Prompt）
outputs	模型输出
tokens	输入/输出 Token 数
latency_ms	耗时（毫秒）
run_type	类型：llm / chain / tool / retriever 等
attachments	附加文件（如图片输入/输出）
tags	用户自定义标签（如 "production", "test"）
metadata	附加元数据（模型版本、温度等）

5.4 安全与隐私

 LangSmith 高度重视数据安全和隐私保护。

- **数据隔离**：每个组织的 Traces 完全隔离
- **敏感信息过滤**：支持配置自动过滤敏感字段（如 API Key、密码等）
- **脱敏工具**：提供 PII 检测和自动替换功能
- **SOC 2 认证**：企业版支持 SOC 2 Type II 合规
- **数据保留策略**：支持自定义数据保留天数，过期自动清理

5.5 定价模式

方案	说明
免费版（Free）	每月有限额的 Trace 数量和评估次数，适合个人开发和小规模实验

标准版（Standard）	按量付费，解锁更多 Trace 额度和协作功能
企业版（Enterprise）	自定义用量、SOC 2、GPDR、SSO、专用支持、私有化部署等

💡 推荐在项目早期就接入 LangSmith，因为 Trace 数据是历史积累的，越早开始积累越有价值。

六、实战场景示例

场景一：线上 Bad Case 排查

1. 用户反馈 AI 助手在某个问题上回答错误
2. 在 LangSmith Dashboard 中按时间范围过滤，找到该用户的最近一次调用
3. 查看完整 Trace：用户输入 → Prompt 拼接 → LLM 调用 → 输出
4. 发现问题出在 Prompt 模板的某个边缘 case 遗漏
5. 修改 Prompt，重新上线，用同一测试集验证修复效果

场景二：Prompt 版本 A/B 测试

6. 在 Prompt Registry 中创建两个版本：v1（当前生产版）和 v2（优化版）
7. 将 10% 的流量导向 v2，90% 保留 v1
8. 观察两个版本的评估分数、用户反馈和 Token 消耗
9. v2 效果优于 v1，逐步将流量切换至 v2

场景三：回归测试

10. 建立包含 100 条典型用例的数据集（Datasets）
11. 每次代码变更（尤其是 Prompt 或模型更换）后，自动触发评估流水线
12. 系统自动对比新旧版本的输出质量，生成 diff 报告
13. 分数下降超过阈值时，自动阻止部署

七、相关资源

- 📖 官方文档：<https://docs.smith.langchain.com/>
- 🐙 GitHub：<https://github.com/langchain-ai/langsmith-sdk>
- 📖 LangChain 文档：<https://docs.langchain.com/>
- 💬 Discord 社区：LangChain / LangSmith 官方 Discord

